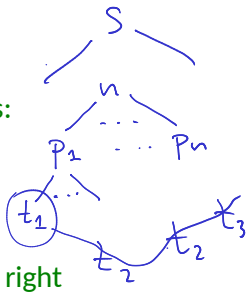
Parse Tree $t \in A$

Given a grammar G = (A, N, S, R), t is a **parse tree** of G
iff t is a node-labelled tree with ordered children that satisfies:

- root is labeled by S
- leaves are labelled by elements of A
- each non-leaf node is labelled by an element of N
- for each non-leaf node labelled by n whose children left to right are labelled by $p_1 \ldots p_n$, we have a rule $(n ::= p_1 \ldots p_n) \in R$



Yield of a parse tree t is the unique word in A^* obtained by reading the leaves of t from left to right

Language of a grammar G = words of all yields of parse trees of G

L(G) = {yield(t) | isParseTree(G,t)}

$w \in L(G) \quad \Leftrightarrow \quad \exists t.\ w = \text{yield}(t) \wedge \text{isParseTree}(G,t)$

isParseTree - **easy** to check condition, given t

Harder: know *if for a word there* ***exists*** *a parse tree*

Grammar Derivation

$S \Rightarrow \underbrace{(S)}_{p_i} \Rightarrow \underbrace{(SS)}_{p_{i+1}} \Rightarrow \underbrace{\quad}_{} \Rightarrow (()())$

A **derivation** for G is any sequence of words $p_i \in (A \cup N)^*$, whose:

- first word is S
- each subsequent word is obtained from the previous one by replacing one of its letters by right-hand side of a rule in R :

  $p_i = \underline{unv}$, $\underbrace{(n::=q)}\in R$,

  $p_{i+1} = uqv$
- Last word has only letters from A

Each parse tree of a grammar has one or more derivations, which result in expanding tree gradually from S

- Different orders of expanding non-terminals may generate the same tree
- Leftmost derivation: always expands leftmost non-terminal
  - Rightmost derivation: always expands rightmost non-terminal

Remark

We abbreviate

$$S ::= p$$

$$S ::= q$$

as

$$S ::= p \mid q$$

Example: Parse Tree vs Derivation

Consider this grammar G = ({a,b}, {S,P,Q}, S, R) where R is:

A N

$P \Rightarrow aP \Rightarrow aaP \Rightarrow aaa$

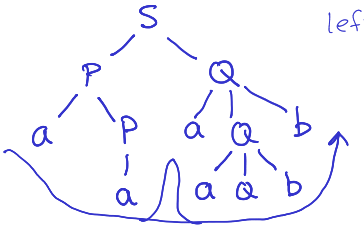
$Q \Rightarrow aQb \Rightarrow a\,aQb\,b \Rightarrow aabb$

R
- S ::= PQ
- P ::= a | aP
- Q ::= ε | aQb

P Q

Show a parse tree for aaaabb

Show at least two derivations that correspond to that tree.



leftmost: $S \Rightarrow PQ \Rightarrow aPQ \Rightarrow aaQ \Rightarrow aaaQb \Rightarrow$
$\Rightarrow aaaaQbb \Rightarrow aaaabb$

rightmost $S \Rightarrow PQ \Rightarrow P\,aQb \Rightarrow P\,a\,aQbb \Rightarrow$
$\Rightarrow P\,aabb \Rightarrow aP\,aabb \Rightarrow \boxed{aaaabb}$

$aaaabb \in L(G)$

Balanced Parentheses Grammar

$A = \{), (\}$

S

$S ::= \varepsilon \mid (S)$

$(())$

$S ::= \varepsilon \mid (S)S \mid S(S)$

Consider the language L consisting of precisely those words consisting of parentheses "**(**" and "**)**" that are balanced (each parenthesis has the matching one)

- Example sequence of parentheses

  ((()) ()) - balanced, belongs to the language

  ( )) (( ) - not balanced, does not belong

  ?!

Exercise: give the grammar and example derivation for the first string.

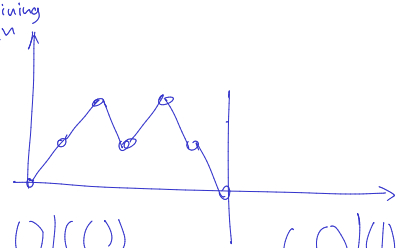
Balanced Parentheses Grammar

G_1 S ::= ε | S(S)S

G_2 S ::= ε | (S)S

G_3 S ::= ε | S(S)

G_4 S ::= ε | S S | (S)



S ⇒ SS ⇒ SSS ⇒ S(S)S

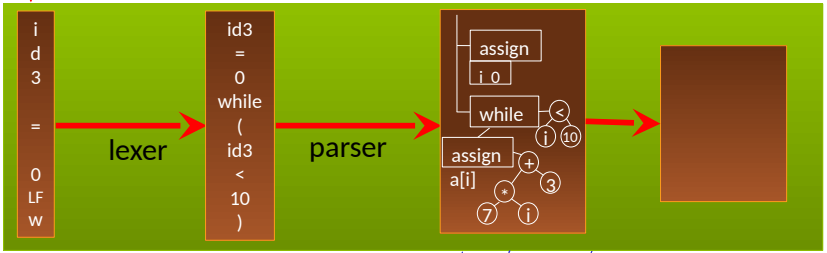
These all define the same language, the language of balanced parentheses.

Parse Trees vs Syntax Trees

```
Id3 = 0
while (id3 < 10) {
  println(“”,id3);
  id3 = id3 + 1 }
```

source code

Compiler



characters

Char

words
(tokens)

A

abstract syntax
trees

AST

While Language Syntax

This syntax is given by a context-free grammar:

program ::= statmt*

statmt ::= println(stringConst , ident)

| ident = expr

| **if** (expr) statmt (else statmt)?

| **while** (expr) statmt

| { statmt* }

expr ::= intLiteral | ident

| expr (&& | < | == | + | - | * | / | %) expr

| ! expr | - expr

$P ::= T^*$

$P ::= T_1$

$T_1 ::= \varepsilon \mid T T_1$

$P ::= T_2$

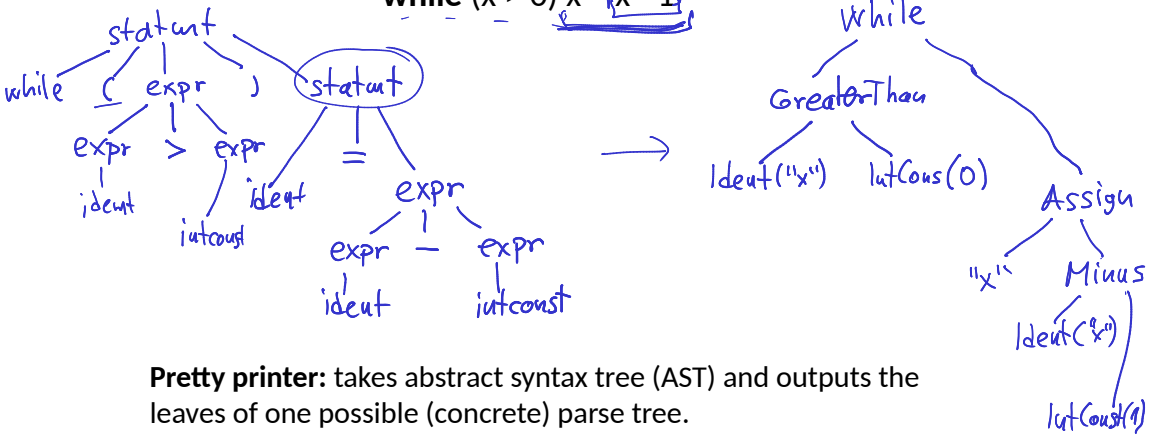
$T_2 ::= \varepsilon \mid T_2 T$

O ::= && | < | == | + | - | * | / | %

expr O expr

Parse Tree vs Abstract Syntax Tree (AST)

while (x > 0) x = $\boxed{\text{x - 1}}$



Pretty printer: takes abstract syntax tree (AST) and outputs the leaves of one possible (concrete) parse tree.

parse(prettyPrint(ast)) ≈ ast

Parse Tree vs Abstract Syntax Tree (AST)

- Each node in **parse tree** has children corresponding **precisely to right-hand side of grammar rules**. The definition of parse trees is fixed given the grammar
  - Often compiler never actually builds parse trees in memory
- Nodes in **abstract syntax tree (AST)** contain only useful information and usually omit the punctuation signs. We can choose our own syntax trees, to make it convenient for both construction in parsing and for later stages of our compiler or interpreter
  - **A compiler often directly builds AST**

Abstract Syntax Trees for Statements

grammar:

statmt ::= println (stringConst , ident)
| ident = expr
| if (expr) statmt (else statmt)?
| while (expr) statmt
| { statmt* }

AST classes:

```
abstract class Statmt
case class PrintlnS(msg : String, var : Identifier) extends Statmt
case class Assignment(left : Identifier, right : Expr) extends Statmt
case class If(cond : Expr, trueBr : Statmt,
                          falseBr : Option[Statmt]) extends Statmt
case class While(cond : Expr, body : Expr) extends Statmt
case class Block(sts : List[Statmt]) extends Statmt
```

Abstract Syntax Trees for Statements

statmt ::= println (stringConst , ident)
| ident = expr
| **if** (expr) statmt (else statmt)?
| **while** (expr) statmt
| { statmt* }

```
abstract class Statmt
case class PrintlnS(msg : String, var : Identifier) extends Statmt
case class Assignment(left : Identifier, right : Expr) extends Statmt
case class If(cond : Expr, trueBr : Statmt,
                          falseBr : Option[Statmt]) extends Statmt
case class While(cond : Expr, body : Statmt) extends Statmt
case class Block(sts : List[Statmt]) extends Statmt
```

While Language with Simple Expressions

statmt ::=

```
        println ( stringConst , ident )
      | ident = expr
      | if ( expr ) statmt (else statmt)?
      | while ( expr ) statmt
      | { statmt* }
```

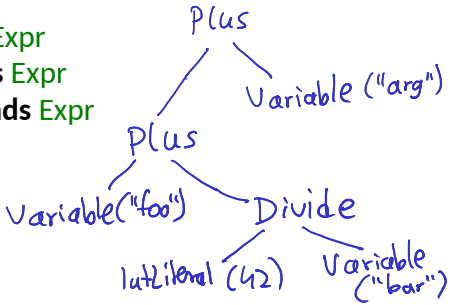
```
expr ::= intLiteral | ident
       | expr ( + | / ) expr
```

Abstract Syntax Trees for Expressions

```
expr ::= intLiteral | ident
       | expr + expr | expr / expr
```

```
abstract class Expr
case class IntLiteral(x : Int) extends Expr
case class Variable(id : Identifier) extends Expr
case class Plus(e1 : Expr, e2 : Expr) extends Expr
case class Divide(e1 : Expr, e2 : Expr) extends Expr
```

(foo + (42 / bar) + arg



Ambiguous Grammars

```
expr ::= intLiteral | ident
       | expr + expr | expr / expr
```

ident + intLiteral / ident + ident

Each node in parse tree is given by one grammar alternative.

Ambiguous grammar: if some token sequence has **multiple parse trees** (then it is has multiple abstract trees).